In Next.js, `router.back()` programmatically triggers a browser history back action, which by design fires a `popstate` event. Because both manual browser navigation and `router.back()` rely on this same underlying mechanism, you cannot "stop" the event from firing entirely; instead, you must **distinguish** between them or use Next.js-specific methods to handle the logic. MDN Web Docs +3

1. Use `router.beforePopState` (Pages Router)

If you are using the **Pages Router**, the most effective way to handle or block specific popstate events is through `router.beforePopState`. This hook allows you to return `false` to prevent Next.js from acting on the navigation. Next.js +1

javascript

```javascript
import { useRouter } from 'next/router';
import { useEffect } from 'react';

const MyComponent = () => {
  const router = useRouter();

  useEffect(() => {
    router.beforePopState(({ url, as, options }) => {
      // You can implement logic here to determine if this popstate
      // should be ignored or handled differently.
      // Returning false prevents the router from handling the change.
      return true;
    });
  }, [router]);
};
```

Use code with caution.

2. Implement a Flag (App or Pages Router)

Since `popstate` doesn't natively tell you if it was triggered by code or a button, you can set a temporary flag when you call `router.back()`. GitHub

- **Step 1:** Create a ref or state variable (e.g., `isProgrammaticBack`).
- **Step 2:** Set it to `true` immediately before calling `router.back()`.
- **Step 3:** In your `handlePopState` function, check the flag. If `true`, ignore the logic and reset the flag.

javascript

```javascript
const isProgrammaticBack = useRef(false);

const handleBackClick = () => {
  isProgrammaticBack.current = true;
  router.back();
};

const handlePopState = (event) => {
  if (isProgrammaticBack.current) {
    isProgrammaticBack.current = false; // Reset for next time
    return; // Stop custom logic for this call
  }
  // Your logic for actual browser back button goes here
};
```

Use code with caution.

3. Alternative: Use `useEffect` with `usePathname` (App Router)

If you simply need to react to navigation changes without low-level event handling, listening to the pathname is often safer in the App Router as it ignores the manual `popstate` mechanics entirely. GitHub +1

javascript

```javascript
import { usePathname } from 'next/navigation';
import { useEffect } from 'react';

function NavigationListener() {
  const pathname = usePathname();

  useEffect(() => {
    // This runs on every successful route change
    console.log('Route changed to:', pathname);
  }, [pathname]);
}
```

Use code with caution.

Summary Comparison

Method	Best For	Pros
`beforePopState`	Pages Router	Official Next.js way to intercept or block.
Ref Flagging	Distinguishing events	Simple way to tell if *you* triggered the back action.
`usePathname`	App Router	Reliable for simple "on navigation" logic.